ТЕХНИЧЕСКИ УНИВЕРСИТЕТ – СОФИЯ

ФАКУЛТЕТ КОМПЮТЪРНИ СИСТЕМИ И ТЕХНОЛОГИИ

ПРОЕКТНА РАЗРАБОТКА

Система за управление на версии на документи
Version Control System “Manage Yourself”

Изготвили: *Ekun Tech Blossoms*

УВОД

Бързите темпове на развитие на информационните технологии налагат необходимостта от по-ефективни и структурирани начини за съхранение, споделяне и управление на информация. В съвременните организации документите са основен носител на знания, процеси и комуникация, а тяхното правилно управление е от съществено значение за ефективната работа на екипите. Често възникват проблеми като загуба на файлове, липса на контрол върху версиите, затруднено сътрудничество и невъзможност за проследяване на направените промени.

В този контекст системите за управление на версии на документи придобиват ключово значение. Те предоставят централизирана среда за работа, възможност за проследяване на промените и по-добра организация на информацията. Въпреки това, много от съществуващите решения са насочени към корпоративни среди и могат да бъдат сложни или неинтуитивни за крайни потребители.

Проектът „Manage Yourself – Version Control System“ е разработен с цел да демонстрира основните принципи на управление на документи и техните версии чрез уеб базирано приложение. Системата има учебен и демонстрационен характер и представя базови функционалности, свързани със създаване на документи, добавяне на версии и съхранение на файлове.

В рамките на разработката е поставен акцент върху създаването на ясен и интуитивен потребителски интерфейс, който да улеснява навигацията и работата с информацията. Освен реалната имплементация, в проекта са разгледани и допълнителни концепции като ролеви модел, контрол на достъпа и разширена архитектура, които показват възможности за бъдещо развитие.

Проектът е разработен като част от практическа подготовка по софтуерно инженерство и цели да демонстрира основни принципи и подходи, използвани при изграждането на уеб приложения. В документацията са описани всички етапи от разработката – от проучването и анализа на съществуващи решения, през проектирането, до реализацията и оценката на системата.

Важно е да се отбележи, че проектът има демонстрационен характер и не всички разгледани функционалности са напълно реализирани. Част от тях са включени с цел илюстриране на добри практики и възможности за бъдещо развитие.

ЦЕЛИ И ЗАДАЧИ

Основната цел на настоящата проектна разработка е създаването на уеб базирана система за управление на версии на документи, която да осигурява централизирано и структурирано съхранение на файлове и техните версии. Системата *Manage Yourself – Version Control System* има за цел да демонстрира основните принципи на работа с документи и проследяване на промените в тях.

За постигане на тази цел са формулирани следните основни задачи:

1. Проектиране на архитектура на системата

- Проектиране на логическа архитектура на приложението, която разделя системата на основни компоненти – потребителски интерфейс, бизнес логика и слой за съхранение на данни.
- Разглеждане на концептуален модел за по-сложна архитектура (включително микросървисен подход), с цел демонстрация на възможности за бъдещо развитие.

2. Разглеждане на модел за контрол на достъпа

В рамките на проекта е анализиран примерен ролеви модел, който включва различни типове потребители (например автор, рецензент и администратор).

Този модел има концептуален характер и цели да покаже как може да бъде организирана система за контрол на достъпа в по-голямо приложение.

3. Разработване на функционалности за управление на документи и версии

Реализиране на основни функционалности, включващи:

- създаване на документи
- качване на файлове
- добавяне на нови версии
- съхранение на информация за версиите

Разглеждане на допълнителни функционалности като сравнение между версии и проследяване на историята като част от концептуалния модел.

4. Създаване на потребителски интерфейс

Разработване на интуитивен интерфейс, който позволява лесна навигация и работа с документите.

Интерфейсът включва основни визуални елементи за представяне на документи, версии и информация за избрания файл.

5. Реализиране на бекенд логика

Изграждане на сървърна логика за обработка на заявки, свързани с документи и версии.

Използване на подходящи технологии за организиране на кода и осигуряване на коректна обработка на данните.

6. Изграждане на модел на данните

Проектиране на структура на данните, включваща основните обекти в системата – потребители, проекти, документи, версии и файлове.

Дефиниране на връзки между тях с цел осигуряване на логическа и последователна организация на информацията.

7. Тестване на основната функционалност

Провеждане на тестове с примерни данни с цел проверка на основните процеси, свързани с работа с документи и версии.

Оценка на коректността на изпълнение на основните операции.

8. Подготовка на документация

Създаване на подробна документация, описваща структурата, функционалностите и концепциите, използвани в проекта.

Включване на диаграми и описания, които представят както реализацията, така и възможностите за бъдещо развитие.

ГЛАВА 1. ПРОУЧВАНЕ ПО ТЕМАТА

1.1. Предпоставка за създаване на продукта

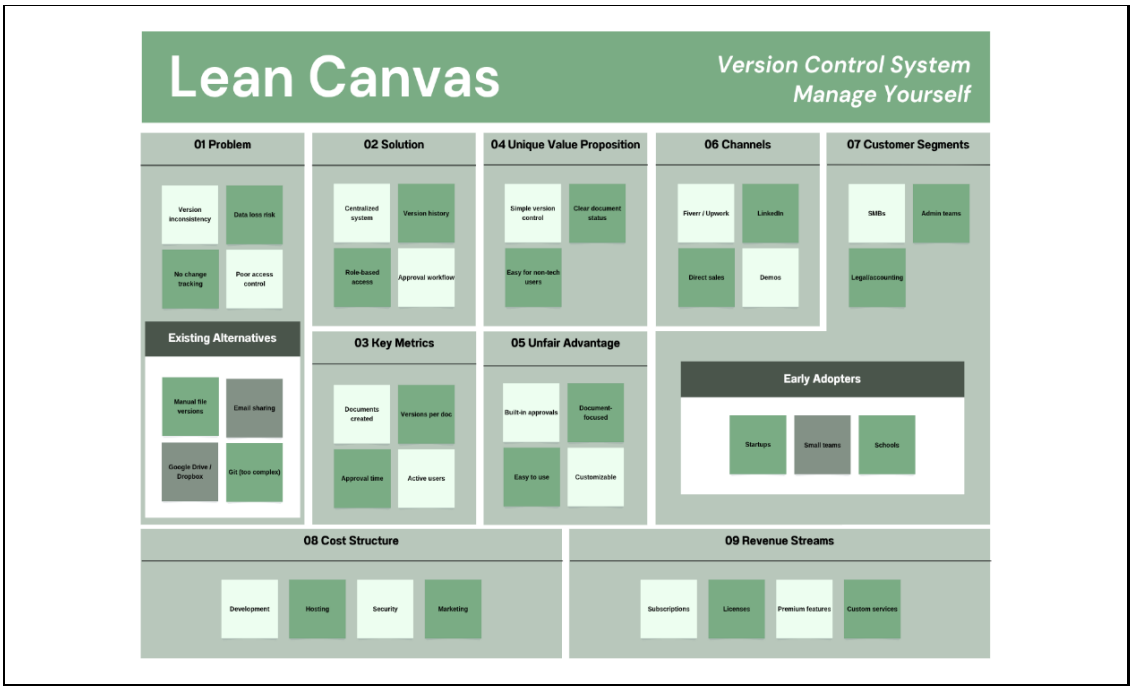
В съвременната дигитална среда управлението на информацията се утвърждава като ключов фактор за ефективността на всяка организация. Документите играят централна роля като носители на знания, бизнес процеси и комуникация, което прави тяхното структуриране, съхранение и проследяване от съществено значение за нормалното функциониране на екипите.

С увеличаването на обема от информация и необходимостта от бърз достъп до актуални данни, традиционните методи за работа с документи постепенно губят своята ефективност. В много случаи все още се използват неструктурирани подходи като изпращане на файлове по електронна поща, съхранение в локални директории или липса на централизирана система за управление. Това води до редица практически затруднения, свързани със загуба на информация, работа върху неактуални версии на файлове, както и възникване на конфликти между различни редакции на един и същ документ.

Липсата на ясна история на промените и механизъм за проследяване на извършените редакции затруднява координацията между членовете на екипа. Освен това, отсъствието на централизирано място за съхранение и контрол допълнително усложнява процеса на съвместна работа и намалява прозрачността при обработката на документи.

В този контекст възниква необходимост от решение, което да осигури по-добра организация на файловете, проследимост на промените и по-ефективна екипна работа. Подобна система следва да позволява централизирано управление на документи, създаване и съхранение на различни версии, както и възможност за проследяване на развитието на даден файл във времето.

Проектът *Manage Yourself – Version Control System* е разработен като учебен прототип, който има за цел да демонстрира основните принципи на управление на версии на документи. Той представя концептуален модел за организиране и проследяване на файлове в екипна среда, като акцентира върху идеята за версия, история на промените и по-добра структура на работния процес. Важно е да се отбележи, че системата има демонстрационен характер и част от разглежданите функционалности са представени на концептуално ниво като възможности за бъдещо развитие.



Фиг. 1 Lean Canvas

1.2. Съществуващи решения и реализации

Съвременният пазар предлага разнообразие от решения за управление на документи и версии, които се различават по своята функционалност, сложност и насоченост към различни групи потребители. Анализът на тези решения показва, че те могат да бъдат разгледани в няколко основни направления.

1.2.1. Облачни платформи за съхранение на файлове

Google Drive, Dropbox, OneDrive

- Тези услуги предоставят удобен начин за съхранение и споделяне на файлове, както и базова поддръжка на версии. Въпреки това, те не предлагат ясно структуриран процес за управление на версиите в контекста на екипна работа, а възможностите за контрол на достъпа и проследяване на промените са ограничени спрямо по-специализирани системи.

1.2.2. Корпоративни системи за документооборот

Microsoft SharePoint, Alfresco, OpenText

- Те предлагат богата функционалност, включително управление на документи, контрол на достъпа и автоматизация на процеси. Въпреки техните възможности, тези системи често са свързани с висока степен на сложност, необходимост от обучение и значителни ресурси за

внедряване и поддръжка, което ги прави по-малко подходящи за малки екипи или учебни проекти.

1.2.3. Системи за контрол на версии за програмен код

Git, GitLab, GitHub

- Те осигуряват изключително мощни механизми за проследяване на промени, работа с версии и екипна колаборация. Въпреки това, тези системи са основно ориентирани към разработчици и работа с програмен код, което ги прави по-трудни за използване от нетехнически потребители и по-малко подходящи за управление на стандартни документи.

1.2.4. Специализирани системи за управление на документи

- Съществуват и специализирани системи за управление на документи, които се стремят да комбинират удобството на облачните услуги с функционалността на корпоративните решения. Въпреки това, част от тях предлагат ограничени възможности или не осигуряват достатъчно гъвкавост и интуитивност при работа.

Извод от анализа

В резултат на този анализ се очертава необходимостта от решение, което да съчетава удобство, яснота и основна функционалност, без да въвежда излишна сложност. Проектът *Manage Yourself* се стреми да отговори именно на тази необходимост, като представя опростен модел за управление на версии на документи. Той не цели да замени съществуващите системи, а да демонстрира как техните основни идеи могат да бъдат приложени в по-достъпен и разбираем вид, подходящ за учебна среда и базови екипни процеси.

ГЛАВА 2. ПРОЕКТИРАНЕ НА СТРУКТУРАТА НА СИСТЕМАТА „MANAGE YOURSELF“

2.1. Функционални изисквания към уеб базирано приложение за управление на версии на документи

Функционалните изисквания описват основните възможности, които една система за управление на версии на документи следва да предоставя. Те са формулирани на база направеното проучване и имат за цел да очертаят както реално реализираните функционалности, така и възможните разширения на системата.

Важно е да се отбележи, че в рамките на проекта част от описаните изисквания са реализирани в опростен вид, докато други са разгледани концептуално с цел демонстрация на пълноценна система.

2.1.1. Роли и права в системата

В документацията е разгледан примерен модел за роли и права, който включва различни типове потребители и нива на достъп. Този модел има за цел да илюстрира как може да бъде организирана една реална система.

В текущата реализация не е изградена пълна система за автентикация и авторизация, поради което ролите имат демонстрационен характер.

2.1.2. Управление на документи

- Системата разглежда концепцията за управление на документи, която включва създаване и организиране на файлове в логическа структура. Потребителят може да работи с документи като основни единици, към които се асоциират различни версии.

Функционалността е представена в опростен вид, като основният фокус е върху организацията и проследяването на файлове, а не върху пълна реализация на документооборотна система.

2.1.3. Управление на версии

Основна концепция в проекта е управлението на версии. За всеки документ се разглежда възможността за създаване на последователни версии, които отразяват развитието на съдържанието във времето.

Системата демонстрира идеи като:

- добавяне на нови версии
- проследяване на историята
- разграничаване между различни състояния на файла

Функционалности като сравнение между версии, както и процеси по одобрение и отхвърляне, са разгледани концептуално и не са напълно реализирани.

2.1.4. Експорт и визуализация

- Системата позволява работа с файлове в техния оригинален формат, без да налага ограничения върху типа съдържание. Основната идея е да се демонстрира управление на файлове, а не тяхното преобразуване.

Функционалности като експорт в специфични формати са разгледани като възможно разширение, но не представляват основна част от текущата реализация.

2.1.5. Управление на потребители и роли

- Концепцията за управление на потребители е разгледана на теоретично ниво, като част от цялостния модел на системата. Описани са възможности като създаване, редактиране и деактивиране на потребители, но тези функционалности не са напълно реализирани.

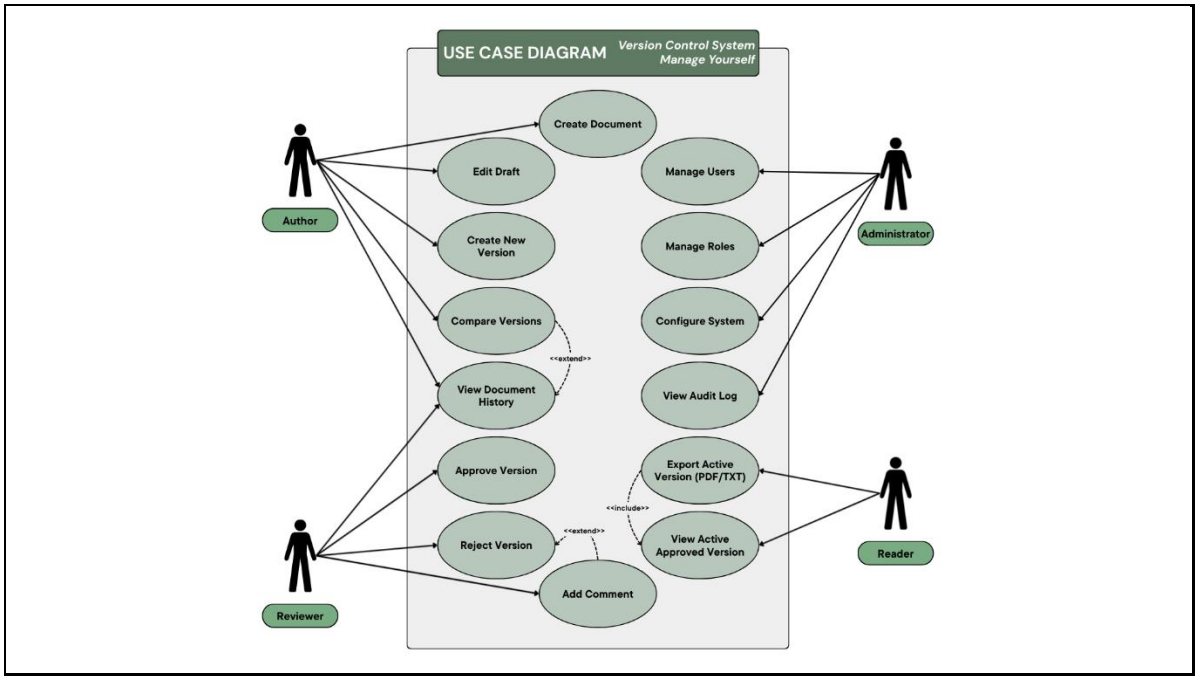
2.1.6. Потребителски профил

- В рамките на системата е предвидена концепция за потребителски профил, който позволява персонализация на потребителските данни. Тази функционалност е разгледана като част от бъдещо развитие на системата.

2.1.7. Системни изисквания

- Проектът разглежда използването на добри практики като сигурност, проследимост и структурираност на данните. Концепции като централизирано съхранение, логване на действия и API комуникация са представени като част от модел на една по-пълна система, но не са изцяло реализирани в текущата версия.

2.2. Изготвяне на Use Case диаграми и описание на случаите на употреба



Фиг.2 Use Case Diagram

Use Case диаграмите описват взаимодействието между потребителя и системата, като представят основните сценарии на работа. В проекта тези сценарии са използвани с цел да се моделира поведението на една пълноценна система.

Важно е да се отбележи, че описаните актьори (Автор, Рецензент, Читател, Администратор) са част от концептуален модел и не са напълно имплементирани като реални роли в системата.

Основни Use Case-и и техните описания

По-долу са представени ключови случаи на употреба, които описват начина, по който една система за управление на версии на документи би могла да функционира. Те имат **концептуален характер** и служат за моделиране на поведението на системата.

Use Case: Създаване на документ

Цел:
Демонстрира процеса по създаване на нов документ, към който могат да бъдат добавяни версии.

Категория: Основна

Предусловия:
Потребителят има достъп до системата (концептуално дефиниран).

Успешен резултат:
Създава се нов документ, който може да бъде използван за добавяне на версии.

Неуспешен резултат:
Възниква грешка при създаване (напр. липсваща информация).

Актьори: Потребител (концептуално: Автор, Администратор)

Use Case: Качване на нова версия

Цел:
Илюстрира добавяне на нова версия към съществуващ документ.

Категория: Основна

Предусловия:
Съществува документ.

Успешен

резултат:
Добавя се нова версия към документа.

Неуспешен резултат:
Файлът не може да бъде обработен.

Актьори: Потребител

Use Case: Сравнение между версии

Цел:
Показва как може да се извършва сравнение между различни версии.

Категория: Разширена

Предусловия:
Налични са поне две версии.

Успешен

резултат:
Потребителят получава информация за разликите между версиите.

Неуспешен резултат:
Грешка при обработка на файловете.

Забележка:
Функционалността е разгледана концептуално.

Актьори: Потребител

Use Case: Одобрение на версия

Цел:
Илюстрира процес на валидиране на версия.

Категория: Разширена

Предусловия:
Съществува версия, подлежаща на оценка.

Успешен

резултат:
Версията се маркира като одобрена (концептуално).

Неуспешен резултат:
Липса на необходимите условия.

Забележка:
Тази функционалност не е реализирана и е представена като възможно разширение.

Актьори: Потребител (концептуално: Рецензент)

Use Case: Отхвърляне на версия

Цел:
Демонстрира процес на отхвърляне на версия.

Категория: Разширена

Предусловия:
Версията подлежи на оценка.

Успешен резултат:
Версията се отбелязва като отхвърлена.

Неуспешен резултат:
Възниква грешка при обработка.

Забележка:
Функционалността е концептуална.

Актьори: Потребител

Use Case: Преглед на версия

Цел:
Потребителят разглежда налична версия на документ.

Категория: Основна

Предусловия:
Съществува поне една версия.

Успешен резултат:
Файлът се визуализира.

Неуспешен резултат:
Файлът не може да бъде зареден.

Актьори: Потребител

Use Case: Експорт

Цел:
Илюстрира възможността за извеждане на файл.

Категория: Разширена

Предусловия:
Съществува файл.

Успешен резултат:
Файлът се предоставя в наличния формат.

Неуспешен резултат:
Грешка при обработка.

Забележка:
Системата не ограничава формата на файловете.

Актьори: Потребител

Use Case: Управление на потребители

Цел:
Описва възможността за управление на потребители.

Категория: Разширена

Предусловия:
Системата поддържа потребители (концептуално).

Успешен резултат:
Промените се отразяват успешно.

Неуспешен резултат:
Грешка при обработка.

Забележка:
Функционалността не е реализирана.

Актьори: Администратор (концептуално)

2.3. Потребителски интерфейс (UI/UX)

Потребителският интерфейс е проектиран с цел да бъде интуитивен и лесен за използване. В рамките на проекта той е представен в **опростен и**

концептуален вид, който илюстрира основните идеи за организация на информацията и взаимодействие със системата.

Основната цел на интерфейса е да осигури ясна структура, в която потребителят може лесно да навигира между документи и версии. Представеният модел демонстрира как би могла да бъде организирана една подобна система, като не всички елементи са напълно реализирани в текущата версия.

Интерфейсът включва основни компоненти като навигационна структура, визуализация на версии и панел с информация за избрания файл. Описанието има за цел да представи логиката на работа, а не задължително реалната имплементация.

2.3.1. Начална страница – Login / Register

Тази страница предоставя възможност за:

- **вход** в системата за регистрирани потребители
- **регистрация** на нови потребители

Формите са минималистични, с ясно обозначени полета за имейл, парола и потребителско име.

Целта е да се осигури бърз достъп до системата, като същевременно се гарантира сигурност чрез валидиране на данните.

2.3.2. Главно табло (Dashboard) – „Manage Yourself“

- Главното табло е разгледано като основна работна среда, в която потребителят взаимодейства със системата. Представеният модел включва разделяне на интерфейса на няколко логически секции, които подпомагат организацията на информацията.
- Лявата част на интерфейса е предназначена за визуализация на структура от документи и папки. Тази структура има за цел да улесни навигацията и логическото групиране на информацията.
- Централната част представя информация, свързана с версии на документи. Тук концептуално се визуализират различни версии, както и основни характеристики като име на файл и последователност на промените. Функционалности като статуси (например „одобрена“ или „отхвърлена“) са включени като част от модел на разширена система.
- Дясната част съдържа допълнителна информация за избрания документ, включително метаданни и история на промените. Някои от описаните

елементи, като списъци с колаборатори или чернови, са представени концептуално.

2.3.3. Страница за редакция на профил

- В модела на системата е включена възможност за управление на потребителски профил. Тази функционалност позволява редакция на основни потребителски данни.

2.3.4. UX принципи, използвани в системата

- При проектирането на интерфейса са взети предвид основни принципи на потребителското изживяване. Те включват консистентност в дизайна, минимализъм на елементите и стремеж към интуитивност при взаимодействието.
- Целта е да се създаде логична и лесна за използване структура, която да подпомага потребителя при работа с документи и версии, независимо от нивото му на техническа подготовка.

2.4. Описание на избраната технология и софтуерните средства

Изборът на технологии и софтуерни средства е важен етап при разработването на всяка софтуерна система, тъй като определя нейната структура, възможности и потенциал за развитие.

В рамките на настоящия проект се използват основно инструменти за управление на версиите и разработка, като част от по-широк модел на система са разгледани и допълнителни технологии на концептуално ниво.

2.4.1. Основни използвани технологии

- Основният инструмент, използван в проекта, е **Git**, който позволява проследяване на промените и управление на версиите на файловете.
- За съхранение и споделяне на проекта е използвана платформата **GitHub**, която предоставя възможност за съвместна работа, управление на репозитории и проследяване на историята на развитието.
- Тези технологии формират основата на проекта и отразяват основната му идея – работа с версии и проследимост на промените.

2.4.2. Frontend технологии

- В проекта са използвани базови уеб технологии за представяне на информацията.
- **HTML5** служи за структуриране на съдържанието и изграждане на страниците.

CSS3 се използва за визуално оформление и създаване на консистентен интерфейс.

- **JavaScript** подпомага динамичното поведение на страниците и взаимодействието с потребителя.
- Тези технологии са избрани поради тяхната широко разпространена употреба и лесна интеграция.

2.4.3. Концептуално разгледани backend технологии

- В документацията са разгледани и технологии, характерни за изграждане на по-сложни уеб системи, като например **Java** и **Spring Boot**.
- Те са представени като пример за това как би могла да бъде реализирана пълноценна система с backend логика, **REST API** и управление на потребители.
- Важно е да се отбележи, че тези технологии не са част от реалната имплементация на проекта, а са включени с образователна цел.

2.4.4. База данни и съхранение

Технологиите са избрани поради:

- **стабилност**
- **широка поддръжка**
- **възможност за мащабиране**
- **сигурност**
- **лесна интеграция**
- **подходящи за микросървисна архитектура**

Те осигуряват надеждна основа за система, която трябва да обработва документи, версии, потребители и сложни процеси по одобрение.

2.4.5. Инструменти за разработка

- За разработката на проекта са използвани стандартни инструменти за програмиране и управление на код.
- Освен GitHub, са използвани среди за разработка като IntelliJ IDEA и Visual Studio Code, които подпомагат писането, тестването и организирането на кода.

2.4.6. Причини за избор на технологиите

- Използваните технологии са подбрани с оглед на тяхната достъпност, лесна употреба и приложимост в учебна среда.
- От друга страна, разгледаните концептуални технологии като Java и Spring Boot са включени с цел да покажат възможна посока за развитие към по-сложна и функционална система.
- Този подход позволява проектът едновременно да демонстрира реална работа с версии чрез Git, както и да представи теоретичен модел на по-пълноценна система за управление на документи.

ГЛАВА 3. ПРОГРАМНА РЕАЛИЗАЦИЯ

3.1. Структура на данните и ER модел

В рамките на проекта *Manage Yourself – Version Control System* е разработен опростен модел за съхранение на данни, представен чрез ER диаграма. Този модел има за цел да демонстрира основните зависимости между документите, версиите и потребителите.

Важно е да се отбележи, че моделът е опростен и е насочен към илюстриране на концепцията за управление на версии, а не към изграждане на пълноценна база данни.

Основните обекти в системата са потребители, проекти, документи, версии и файлове, като между тях са дефинирани логически връзки.

3.1.1. Users (Потребители)

Тази таблица съдържа информация за потребителите в системата.

- Основните атрибути включват уникален идентификатор, потребителско име, имейл и парола, както и допълнителна информация като дата на създаване и профилни данни.

Потребителите участват в създаването на документи и версии, което позволява проследяване на авторството.

3.1.2. Projects (Проекти)

Проектите представляват логическа структура за групиране на документи.

- Всеки проект има име, описание, дата на създаване и информация за създателя.

Те служат като контейнер за документи и улесняват организацията на информацията.

Между проектите и документите съществува връзка от тип „един към много“, при която един проект може да съдържа множество документи.

3.1.3. Documents (Документи)

- Документите са основна единица в системата. Всеки документ принадлежи към даден проект и съдържа основна информация като име, описание и дата на създаване.
- Документът служи като контейнер за версии, които отразяват неговото развитие във времето.
- Между документите и версиите е реализирана връзка от тип „един към много“.

3.1.4. Versions (Версии)

- Версиите представляват различни състояния на даден документ. Всяка версия е свързана с конкретен документ и съдържа информация като номер на версия, описание (message) и дата на създаване.

Чрез тази структура се осигурява възможност за проследяване на промените и развитието на документа.

3.1.5. Files (Файлове)

- Таблицата с файлове съдържа информация за реалното съдържание на версиите. Всеки файл е асоциран с конкретна версия и съдържа данни като име, формат и времеви маркер.

Това позволява съхраняване на различни файлове към отделните версии, без ограничение на формата.

3.1.6. Връзки между обектите

ER моделът на системата дефинира следните основни зависимости:

- Един потребител може да създава множество документи
- Един проект може да съдържа множество документи
- Един документ може да има множество версии
- Всяка версия може да има асоциран файл

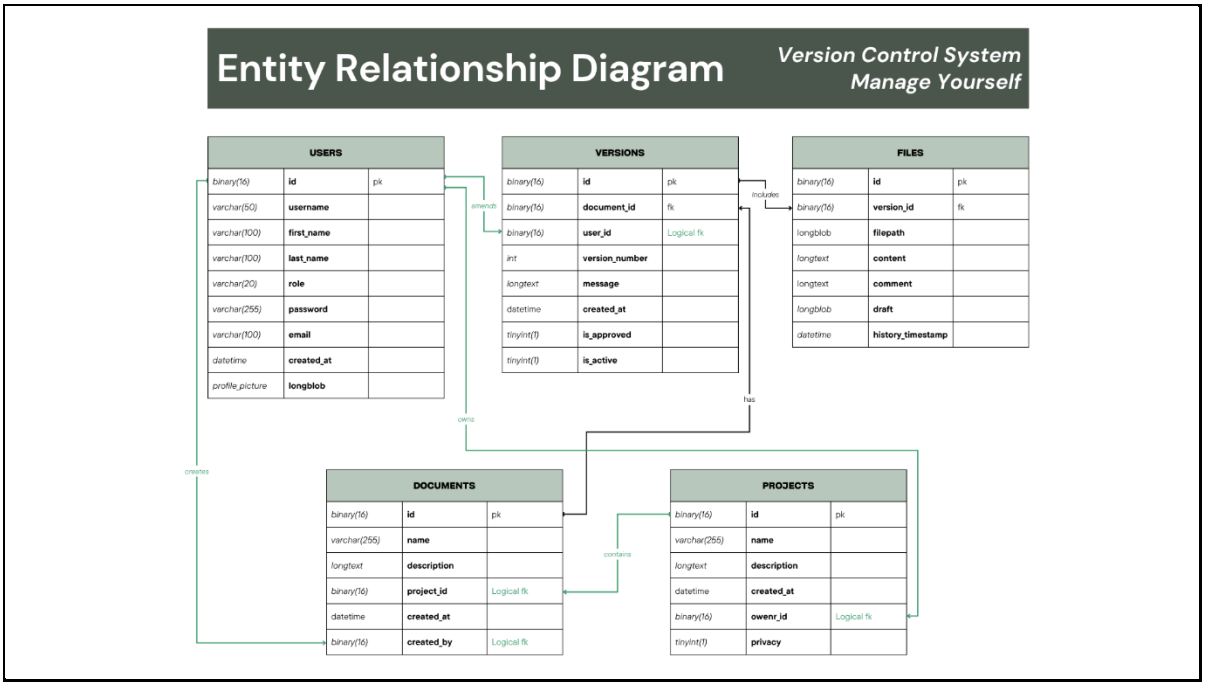
Тези връзки изграждат логическата структура на системата и позволяват проследяване на зависимостите между различните елементи.

3.1.7. Характеристики на модела

Предложеният модел се отличава със своята простота и яснота. Той позволява лесно разбиране на основната идея за управление на версии, като същевременно остава достатъчно гъвкав за бъдещо разширяване.

Липсата на допълнителни компоненти като роли, коментари и логове е умишлена, тъй като системата има демонстрационен характер.

3.1.8. ER Диаграма

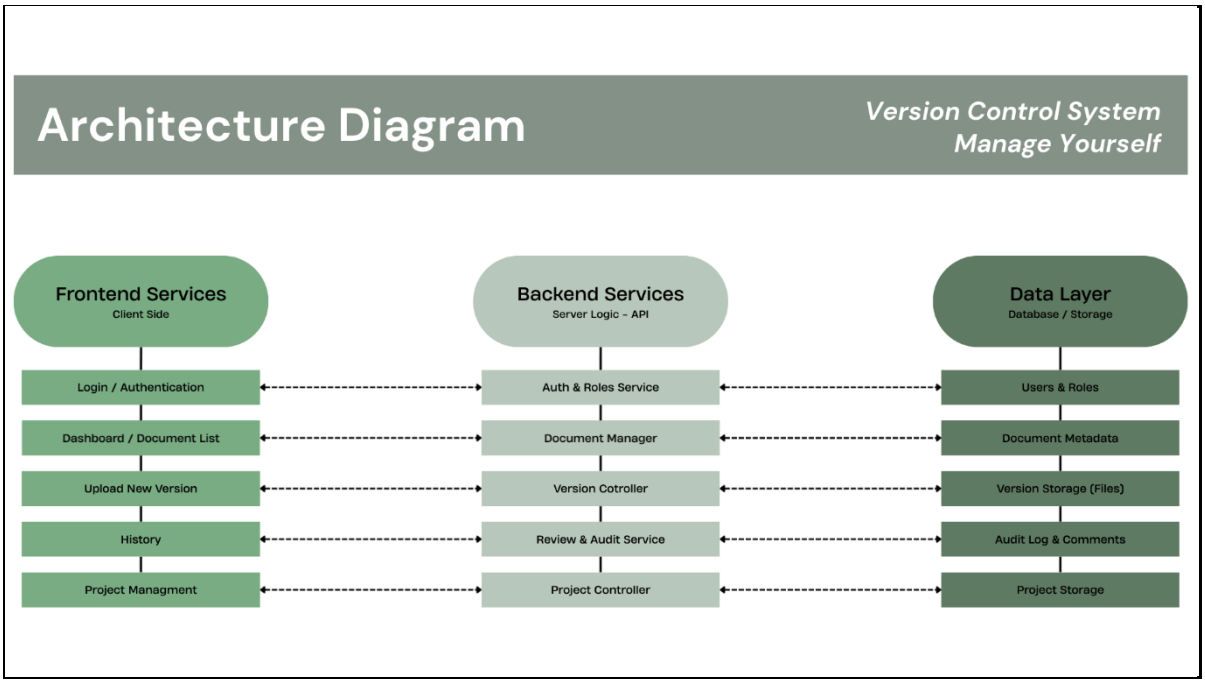


Фиг. 3 Logical ER Diagram

3.2. Структура на приложението

Структурата на приложението *Manage Yourself – Version Control System* е представена чрез архитектурен модел, който илюстрира как би могла да бъде организирана една пълноценна система за управление на версии на документи.

Важно е да се отбележи, че показаната архитектура има **концептуален характер**. Тя служи за демонстрация на добри практики при проектиране на софтуерни системи.



Фиг. 4 Architecture Diagram

3.2.1. Общ архитектурен модел

Архитектурата е организирана в три основни слоя:

- Frontend слой (потребителски интерфейс)
- Backend слой (логика на приложението)
- Data слой (съхранение на данни)

Това разделение позволява ясно разграничаване на отговорностите и улеснява разширяването на системата.

3.2.2. Frontend слой

Frontend частта отговаря за визуализацията и взаимодействието с потребителя.

- В модела са включени основни функционални компоненти като вход в системата, управление на документи, качване на версии и преглед на история.

Тези компоненти са представени като отделни модули, които илюстрират различните действия на потребителя. В реалната имплементация те са реализирани в опростен вид или частично.

Използвани са стандартни уеб технологии като HTML, CSS и JavaScript, които осигуряват базова функционалност и визуализация.

3.2.3. Backend слой

В архитектурната диаграма backend слоят е представен като набор от отделни услуги, които изпълняват различни функции, свързани с обработката на данни.

Тези услуги включват управление на документи, версии и проекти, както и обработка на заявки от потребителя. Разделянето им на отделни компоненти има за цел да покаже как може да се организира логиката в една по-голяма система.

Функционалности като управление на потребители, одобрение на версии и логове са включени в модела като пример за бъдещо развитие.

3.2.4. Data слой

Data слоят е представен като централизирано място за съхранение на информацията.

- В модела са включени данни за потребители, документи, версии и файлове.

Както беше разгледано в предходната точка (ER диаграмата), реалният модел на данните е опростен и включва ограничен брой обекти.

Допълнителни елементи като роли, логове и коментари са представени в архитектурната диаграма като част от разширен модел, но не са реализирани.

3.2.5. Комуникация между компонентите

В архитектурния модел е предвидена комуникация между слоевете чрез API заявки. Това позволява отделяне на потребителския интерфейс от логиката на приложението.

Компоненти като API Gateway и Service Discovery са включени в диаграмата като пример за модерна архитектура.

3.2.6. Интерпретация на архитектурната диаграма

Представената архитектурна диаграма визуализира взаимодействието между трите основни слоя.

- Frontend слоят включва основните действия на потребителя, като работа с документи и версии.
- Backend слоят е разделен на логически компоненти, които обработват тези действия.

- Data слоят осигурява съхранението на информацията.

Диаграмата демонстрира как една система може да бъде структурирана по модулен начин, като същевременно остава гъвкава и разширяема.

3.2.7. Обобщение

Предложеният архитектурен модел представя съвременен подход към изграждане на уеб базирани системи. Въпреки че не е напълно реализиран в рамките на проекта, той показва разбиране за разделяне на слоеве, модулност и възможности за разширение.

Този подход позволява системата да бъде разглеждана не само като текуща реализация, но и като основа за бъдещо развитие.

3.3. Авторизация и оторизация на потребителите на приложението

Сигурността е съществен аспект при разработването на системи, които работят с документи и информация. В рамките на проекта *Manage Yourself – Version Control System* механизмите за авторизация и оторизация са разгледани **на концептуално ниво**, като част от модел на една по-пълноценна система.

Важно е да се отбележи, че в текущата реализация не е изградена пълна система за удостоверяване на потребители и контрол на достъпа.

3.3.1. Авторизация (Authentication)

Авторизацията представлява процесът на удостоверяване на самоличността на потребителя. В модела на системата е предвидено използване на стандартни подходи като вход с потребителско име и парола.

Допълнително е разгледана възможността за използване на токени (например JWT), които позволяват сигурна комуникация между клиент и сървър.

3.3.2. Оторизация (Authorization)

Оторизацията определя какви действия може да извършва даден потребител след успешна идентификация. В рамките на проекта е разгледан примерен модел за контрол на достъпа, който демонстрира как може да се ограничи достъпът до определени функционалности.

3.3.3. Концептуален ролеви модел

В документацията е представен примерен ролеви модел, който включва различни типове потребители с различни права.

Този модел има за цел да илюстрира как би могла да бъде организирана една система за управление на достъпа, като включва роли като администратор, автор, рецензент и читател.

Важно е да се подчертае, че този ролеви модел не е реализиран в текущата версия на системата и служи единствено за демонстрационни цели.

3.3.4. Контрол на достъпа

В една реална система контролът на достъпа може да се реализира чрез проверка на правата на потребителя при всяка заявка. Това позволява ограничаване на действията спрямо ролята на потребителя.

3.3.5. Защита на данните

В рамките на анализа са разгледани и основни принципи за защита на данните, като криптиране на пароли, валидиране на входни данни и защита от често срещани уязвимости.

Тези механизми са характерни за реални системи, но в настоящия проект са представени като насоки за бъдещо развитие.

3.3.6. Проследимост на действията

Възможността за проследяване на действията на потребителите е важен аспект от сигурността. В модела е разгледана идея за записване на ключови действия с цел анализ и прозрачност.

Тази функционалност не е реализирана, но е включена като пример за разширение на системата.

ЗАКЛЮЧЕНИЕ

Разработването на системата *Manage Yourself – Version Control System* представлява цялостен процес, който обхваща проучване, проектиране и реализиране на уеб базирано приложение, насочено към управление на документи и техните версии. В съвременната дигитална среда необходимостта от подобни решения е съществена, тъй като ефективната организация и проследимост на информацията са ключови за работата на екипите.

В рамките на проекта е реализирана опростена система, която демонстрира основните принципи на управление на версии. Основният фокус е поставен върху създаването на документи, добавянето на версии и съхранението на файлове, като се осигурява базова проследимост на промените. Реализацията е с учебен характер и има за цел да илюстрира концепцията за version control при работа с документи.

Архитектурата на системата е опростена и не следва напълно моделите на сложни enterprise решения. Въпреки това, в документацията са разгледани разширени архитектурни и функционални модели, които показват възможни насоки за развитие, включително използване на микросървиси, ролеви модел и механизми за сигурност.

Потребителският интерфейс е проектиран с цел да бъде ясен и интуитивен, като предоставя основни средства за навигация и работа с документи. Чрез него потребителят може лесно да управлява съдържанието и да проследява различните версии.

Разработеният модел на базата от данни осигурява структурирано съхранение на информацията и ясно дефинирани връзки между основните обекти в системата – потребители, проекти, документи, версии и файлове. Това позволява лесно разширяване и надграждане на функционалността.

В заключение, проектът демонстрира основни знания и умения в областта на уеб разработката, базите от данни и софтуерното проектиране. Въпреки своята опростеност, системата може да бъде разглеждана като основа за бъдещо развитие, включващо добавяне на по-сложни механизми като управление на потребители, контрол на достъпа, сравнение на версии и колаборация между потребители.